

SOFTAIMS · PERFORMANCE ENGINEERING

Which load testing tool should we use?

A story, then an equation. Follow one engineer through one week, and the answer becomes arithmetic that anyone in the room can check.

Every technical word is defined where it appears September 2026

No jargon without a definition

Performance testing hides simple ideas behind hard words. Every term in this deck carries its meaning beside it, like this:

Load testing — *sending fake traffic at your system on purpose, to see how it behaves before real customers do*

Virtual user (VU) — *a simulated person, doing one action at a time, over and over*

Throughput — *how much work the system completes per second*

PART ONE

The story

One engineer, one request from her manager, one week. Seven steps from “can it handle 500 users?” to an answer she can defend.

PART TWO

The equation

The story gives us numbers. The numbers go into one formula. We solve it together on screen — and we show exactly when it gives the opposite answer.

MONDAY, 9:15AM

The request lands

SARA'S MANAGER

“The client says the app must handle **500 users**. Can it? I need an answer by Friday.”

Sara is a QA engineer at Softaims

She tests a coaching app: users log in, read their feed, open a conversation, send a message. Ordinary web application, ordinary team.

She has a laptop, a deadline, and one sentence of requirements.

The problem with that sentence

“**500 users**” is not something a computer can send.

It is a description of people, not of traffic. Before Sara can test anything, she must turn a population into numbers a machine can generate.

THIS IS WHERE MOST LOAD TESTING GOES WRONG

Not in the tool. In this translation step — before any tool is opened.

1 TRANSLATE 500 users is not 500 anything

REGISTERED USERS

5,000

Registered — rows in the database — most are asleep

ONLINE AT ONCE

500

Concurrent — people with the app open at the same moment

ACTUAL TRAFFIC

50/s

Requests per second — how many calls actually hit the server each second

How Sara gets from 500 to 50

Each user acts about 6 times per minute
(opens a screen, taps, reads, taps)

$500 \text{ users} \times 6 \text{ actions} = 3,000 \text{ per minute}$
 $3,000 \div 60 \text{ seconds} = 50 \text{ requests/second}$

Between taps, a person *reads*. That pause is called **think time** — the gap between one action and the next — roughly 10 seconds here.

THE NUMBER THAT SURPRISES EVERYONE

At any single instant, only about **10 requests** are actually being processed. The other 490 users are reading their screens.

Building the system for 500 simultaneous requests would over-build it **50 times over**. Sara's first deliverable is not a test result — it is this correction.

Two tools, two very different machines inside

JMETER • BUILT 1998

1 virtual user
↓
1 **operating-system thread**

Thread — *a worker the operating system creates and must keep track of — each one costs real memory*

Simple to picture: one simulated person, one worker. But the laptop pays for every single one.

K6 • BUILT 2016

1,000 virtual users
↓
a handful of **shared workers**

Shared workers — *because users spend most of their time waiting, a few workers can serve many of them by taking turns*

Users spend 98% of their time waiting. k6 exploits that; JMeter reserves a full worker anyway.

WE MEASURED THIS ON A REAL LAPTOP — 1,000 VIRTUAL USERS

JMeter created **1,037 workers**. k6 created **15**. Same simulated users, same work done. That single design choice drives everything on the next three slides.

3 RUN IT Sara's laptop is doing the work too

The tool is not a neutral observer. It is a program on her machine, competing for the same memory and processor.

MEMORY USED

841 vs 320

megabytes — k6 uses 2.6× less

WORKERS CREATED

1,037 vs 15

k6 uses 69× fewer

PROCESSOR TIME

28.2 vs 21.1

seconds — k6 uses 1.34× less

Memory — JMeter		841 MB
Memory — k6		320 MB
Workers — JMeter		1,037
Workers — k6		15

WHY SARA SHOULD CARE

If the tool exhausts her laptop, it **cannot send the traffic it promised**. The server never receives the intended load, and every number she reports is wrong — usually in the flattering direction.

This applies to both tools. It simply happens to JMeter sooner.

A test that quietly lies when things go wrong

What Sara's first test did

Sara asked for 1,000 requests/second.

The server got slow:
200ms → 500ms → 1s → 3s

What the test actually sent:
1000 → 850 → 600 → 300

Her simulated users politely *waited* for a reply before asking again. So the test eased off exactly when the system was suffering.

What real customers do

They arrive when they arrive.

They do not wait for the person
in front of them to be served.

Load stays at 1,000/second
whether you are coping or not.

Matching that behaviour needs an **arrival-rate model** — *start new requests on a clock, regardless of whether earlier ones finished.*

THIS HAS A NAME — COORDINATED OMISSION

Coordinated omission — *the test stops taking measurements during exactly the moments the system is worst, so the report looks better than reality*

Both tools can avoid it — JMeter with its Open Model Thread Group, k6 with arrival-rate scenarios. **Neither avoids it by default.** The difference is how easy it is to get right, and k6 makes it the obvious path.

The average is the most misleading number on the page

100 requests from Sara's run

95 requests	140–160 ms	(fine)
5 requests	2.8–3.2 s	(awful)

Average	292 ms
Median	150 ms
95th percentile	292 ms
99th percentile	3,101 ms

THE AVERAGE DESCRIBES NOBODY

The average says 292 ms. But **only 5 of the 100 users were slower than that**. Ninety-five had a much faster experience, five had a far worse one. **No single user experienced 292 ms.**

Percentile — *line up every response slowest-to-fastest; the 95th percentile is the value 95% of users beat*

This is why performance targets are written as `p95 < 500ms` and `p99 < 1s` — never as an average.

BOTH TOOLS COMPUTE THIS CORRECTLY

This slide is not a difference between the tools. It is the difference between a report that helps and a report that misleads — and it is Sara's judgement, not the software's.

Wednesday: the test has to run forever, not once

A one-off answer expires the moment someone merges new code. Sara needs the test to run automatically on every change.

The rule, written as code

```
thresholds: {  
  http_req_duration: ['p(95)<500'],  
  http_req_failed:   ['rate<0.01']  
}
```

In plain words: *95% of requests must finish under half a second, and fewer than 1% may fail.*

Threshold — a pass/fail rule the tool checks itself — if it breaks, the tool reports failure

What happens on every code change

```
developer pushes code  
↓  
CI pipeline  
↓  
test runs  
↓  
threshold broken  
↓  
build blocked
```

WHY THIS IS THE BIGGEST SINGLE FACTOR

CI/CD — the robot that builds and checks every code change automatically, before it reaches customers

A slowdown caught here costs minutes. The same slowdown found by a customer costs an incident. JMeter can do this too — but the plumbing is something you build, not something you get.

Friday: can a colleague review what Sara wrote?

A JMeter test, as stored on disk

```
<stringProp name="ThreadGroup
  .num_threads">1000</stringProp>
<boolProp name="LoopController
  .continue_forever">false</boolProp>
<elementProp name="HTTPSampler
  .Arguments" elementType="Arg...
```

Roughly 800 lines of **XML** — *a machine-written text format — technically readable, practically not generated by the tool.*

A teammate approves it because it ran, not because they read it.

A k6 test, as stored on disk

```
export default function () {
  const res = http.get(url);
  check(res, {
    'status is 200': r => r.status === 200
  });
  sleep(1);
}
```

About 50 lines of JavaScript — the same language much of the team already writes.

It goes through a **pull request** — *the normal review step where a colleague reads a change before it is accepted* like any other code.

SARA'S CONCLUSION AFTER ONE WEEK

“A test nobody can review is a test nobody can trust. In three years this file will outlive me, and someone will have to change it.”

BUT

There is one thing Sara's new tool simply cannot do

JMeter can test these directly

JDBC	talk straight to a database
JMS	enterprise message queues
LDAP	corporate directory servers
FTP	file transfer
SMTP	mail servers

Protocol — *the language two computers use to talk — a tool can only test the languages it speaks*

k6 speaks the modern web

HTTP / HTTPS	ordinary web APIs
GraphQL	flexible query APIs
gRPC	service-to-service calls
WebSocket	live, always-open connections
Browser	a real Chrome window

Official add-ons now cover MySQL and Postgres too — but JMeter remains far broader.

THIS IS THE HONEST LIMIT — AND IT IS THE WHOLE BASIS OF THE EQUATION

If Softaim's mostly tested databases and message queues, **the recommendation would be JMeter**. So the real question is not “which tool is better?” It is: **how much of our work needs those old protocols?**

Turning the story into one line of arithmetic

$$\text{Fit} = (M \times s_{\text{modern}}) + (L \times s_{\text{legacy}})$$

M = share of our work that is modern web / API · L = share that needs old protocols · s = how well the tool scores on that kind of work, out of 10

Because $M + L = 1$, there is only one unknown. Agree what fraction of our work is legacy, and the answer computes itself.

STEP A

Weights

The room agrees M and L from our real project list

STEP B

Scores

Taken from the story — measured, not opinion

STEP C

Solve

Multiply, add, compare. Two lines of arithmetic

WHY THIS SETTLES THE ARGUMENT

Nobody has to accept my opinion. **You choose the weights.** The conclusion then follows from your own description of Softaims' work — and slide 16 shows precisely when it flips the other way.

What kind of work do we actually do?

KIND OF PROJECT	SHARE	BUCKET
REST / web APIs	45%	Modern
CI performance gates	20%	Modern
GraphQL, gRPC, WebSocket	15%	Modern
Cloud / microservices	10%	Modern
GUI-only QA workflows	3%	Legacy
JDBC / JMS / LDAP / FTP	7%	Legacy
Total	100%	

M — MODERN SHARE

0.90 $45 + 20 + 15 + 10 = 90\%$

L — LEGACY SHARE

0.10 $3 + 7 = 10\%$

CHALLENGE THESE NUMBERS NOW, NOT LATER

This is the one input that is *ours*, not measured. If the room believes legacy work is 30% rather than 10%, say so — the equation will happily give the other answer, and slide 16 shows exactly where the line sits.

The scores come from the story, not from opinion

WORK TYPE	K6	JMETER	EARNED BY
Modern web APIs, CI, cloud	9.5	7.0	Both work correctly. k6 leads on measured resource use (slide 6), reviewable tests (slide 10) and built-in CI thresholds (slide 9). JMeter does all of it — with more effort.
Legacy JDBC, JMS, LDAP, FTP	2.0	10.0	k6 mostly cannot speak these protocols (slide 11). It scores 2, not 0, only because official database add-ons now exist. JMeter is purpose-built for all of them.

DELIBERATELY GENEROUS TO JMETER

JMeter scores **7 out of 10 on modern work** — a clear pass. We are not claiming it fails at web APIs; it does not. And it takes a **perfect 10** on legacy, the highest score anywhere in this deck.

THE ONE ASYMMETRY, AND WHY IT IS FAIR

k6 scores 2 on legacy because of a capability gap, not a preference. A tool that cannot speak a protocol cannot test it at any level of effort. That is the only place either tool is scored near zero.

Two lines of arithmetic

k6

$$\begin{aligned}\text{Fit} &= (0.90 \times 9.5) + (0.10 \times 2.0) \\ &= 8.55 + 0.20 \\ &= 8.75\end{aligned}$$

JMeter

$$\begin{aligned}\text{Fit} &= (0.90 \times 7.0) + (0.10 \times 10.0) \\ &= 6.30 + 1.00 \\ &= 7.30\end{aligned}$$

RESULT

8.75 vs 7.30

k6 wins by **1.45 points** — roughly a **20% better fit** for the work Softaims actually does. Not because JMeter is bad, but because 90% of our work sits where k6 is stronger.

k6



8.75

JMeter



7.30

CHECK THE ARITHMETIC YOURSELF

Nine tenths of ninety-five, plus one tenth of twenty. There is nothing hidden in it — and if you disagree with a weight or a score, change it and the equation still works.

Where this equation gives the opposite answer

An equation that can never say “JMeter” would be a sales pitch. Here is exactly where it does.

LEGACY SHARE (L)	K6	JMETER	WINNER
0%	9.50	7.00	k6
10% ← Softaims	8.75	7.30	k6
23.8%	7.71	7.71	Tie
35%	6.88	8.05	JMeter
50%	5.75	8.50	JMeter
100%	2.00	10.00	JMeter

BREAK-EVEN POINT

23.8%

Above this share of legacy-protocol work, JMeter becomes the better default

THE SINGLE QUESTION THAT DECIDES IT

Is more than **one job in four** at Softaims a database, message-queue or directory test?

If yes — choose JMeter, and this deck was wrong. If no — the arithmetic chooses k6, and it is not close.

The same conclusion in hardware terms

The fit score involves judgement. This one does not — it is division, using memory we measured.

$$\text{Machines needed} = \text{Virtual users required} \div \text{Virtual users per machine}$$

Using measured memory per virtual user, on a 4 GB build server with 30% kept free for safety

k6 — 0.17 MB per virtual user

Usable memory = 2,800 MB
Per virtual user = 0.17 MB

$2,800 \div 0.17 \approx 16,000$ users
on one machine

JMeter — 0.76 MB per virtual user

Usable memory = 2,800 MB
Per virtual user = 0.76 MB

$2,800 \div 0.76 \approx 3,700$ users
on one machine

WHAT THAT MEANS IN MACHINES

To simulate **10,000 users**: k6 needs **one** build server. JMeter needs **three** — plus the work of splitting the test across them, keeping their clocks aligned, and merging the results afterwards.

Caveat, stated plainly: this is a memory calculation only. Processor limits usually bite before memory does, so treat these as ceilings, not promises. The **ratio** is the durable finding.

k6 becomes the default. JMeter stays for what only it can do.

DEFAULT FOR NEW WORK

k6

Web APIs, GraphQL, gRPC, WebSocket, cloud services, and every performance check that runs in CI.

- Fit score **8.75** against JMeter's 7.30
- 2.6× less memory, 69× fewer workers — measured
- Tests reviewed like ordinary code
- Pass/fail rules that block a bad build automatically

KEPT DELIBERATELY

JMeter

Databases, message queues, directory and mail servers — plus every test suite we already have that works.

- Scores a perfect **10** on legacy protocols
- No migration project; nothing gets rewritten
- Stays available for colleagues who prefer the visual builder
- Twenty years of institutional knowledge retained

THE SENTENCE TO REMEMBER

k6 is the strategic default. JMeter is the specialist tool. We are not replacing anything — we are choosing what new work starts with.

Glossary

Load testing — *sending fake traffic on purpose to see how a system copes before customers do*

Virtual user (VU) — *a simulated person, doing one action at a time, repeatedly*

Registered user — *a row in the database; says nothing about traffic*

Concurrent user — *someone with the app open right now*

Request — *one call from the app to the server*

Requests per second (RPS) — *how many calls hit the server each second — the real measure of load*

Throughput — *how much work the system finishes per second*

Response time — *how long one request took, measured from the customer's side*

Latency — *strictly, the network travel time; often used loosely for response time*

Think time — *the pause while a real person reads the screen before tapping again*

Iteration — *one full pass by a virtual user through the whole journey*

Percentile — *line responses up slowest to fastest; p95 is the value 95% of users beat*

Average — *the arithmetic mean — often describes nobody's real experience*

Error rate — *the share of requests that failed, as a percentage*

Thread — *a worker the operating system creates; each one costs memory*

Load generator — *the machine running the test — it has limits too*

Coordinated omission — *when a test stops measuring during the worst moments, making results look better than reality*

Closed model — *users wait for a reply before asking again; eases off when the system struggles*

Open / arrival-rate model — *requests start on a clock regardless of replies; behaves like real customers*

Threshold — *a pass/fail rule the tool checks itself, e.g. p95 under 500 ms*

SLO — *service level objective — the performance target we promise ourselves*

CI/CD — *the robot that builds and checks every code change automatically*

Pull request — *the review step where a colleague reads a change before it is accepted*

Protocol — *the language two computers use to talk; a tool can only test what it speaks*

JDBC — *the protocol for talking directly to a database*

JMS — *enterprise message queueing — systems passing jobs to each other*

LDAP — *corporate directory servers, typically holding staff accounts*

XML — *a machine-written text format; technically readable, practically not*

Sources

Measured by us — reproducible

- **In-house benchmark.** 1,000 virtual users against a target with a known, fixed 50 ms response. Memory, workers and processor time sampled every half second.
`benchmarks/` — harness, raw results and README
- Windows 11 · 8 logical cores · 15.6 GB RAM · k6 v2.1.0 · JMeter 5.6.3 · OpenJDK 17. Both runs completed with 0% errors.
- **Load Testing Mathematics & Engineering** — our 44-page internal reference covering every formula used here.
`benchmarks/Load_Testing_Mathematics_and_Engineering.pdf`

Repository statistics

- GitHub API, retrieved 2 September 2026. k6 31,368 stars; JMeter 9,523. Latest k6 release August 2026; latest JMeter release January 2024.
`api.github.com/repos/grafana/k6` · `api.github.com/repos/apache/jmeter`

Official documentation

- Grafana k6 — open and closed workload models
`grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed/`
- Grafana k6 — thresholds as pass/fail criteria
`grafana.com/docs/k6/latest/using-k6/thresholds/`
- Grafana k6 — built-in metric definitions
`grafana.com/docs/k6/latest/using-k6/metrics/reference/`
- Grafana k6 — memory per virtual user guidance
`grafana.com/docs/k6/latest/set-up/fine-tune-os/`
- Apache JMeter — best practices: non-GUI mode, listeners, thread sizing, coordinated omission
`jmeter.apache.org/usermanual/best-practices.html`
- Apache JMeter — glossary: Elapsed Time and Latency definitions
`jmeter.apache.org/usermanual/glossary.html`
- Apache JMeter — elements of a test plan, protocol coverage
`jmeter.apache.org/usermanual/test_plan.html`

The workload shares on slide 13 are **our own estimate** of Softaims' project mix, not a measurement. They are the one input open to challenge — which is why slide 16 exists.

IN ONE SLIDE

Sara spent a week learning that the hard part was never the tool.

SHE LEARNED

Translate first

500 users meant 50 requests a second and 10 at a time. Getting that wrong dwarfs any tool difference.

SHE LEARNED

Watch the tester

A load generator out of memory reports comfortable numbers about a server it never reached.

SHE LEARNED

Make it repeat

An answer that only exists on Friday is worth little by Monday. It belongs in the pipeline.

8.75 vs **7.30**

And when she put the week into one equation, the arithmetic agreed with her: **k6 for new work, JMeter for the protocols only it can reach** — unless more than one job in four turns out to be legacy.